Minimum-Cost Activity Scheduling for a Cranky Toddler

PROJECT REPORT

Gedela Sai Shashank, Bmat2414
Gudavalli Sathya Tej, Bmat2415



Design and Analysis of Algorithms

October 26, 2026

1 Problem Statement

We are given a set of n activities, where activity k has a start time s_k, an end time e_k with $s_k < e_k$, and an enrollment fee $c_k > 0$. Activity k occupies the closed interval $[s_k, e_k]$. We are also given a set of m cranky times $T = \{t_1 < t_2 < \cdots < t_m\}$, the moments at which the child must be enrolled in some activity.

Definition 1 (Feasible schedule)**.** A set S of activities is a *feasible schedule* if it satisfies both of the following conditions.

1. **Coverage:** the child is enrolled in some activity at every cranky time, that is,

$$\forall\, t \in T \;\; \exists\, k \in S : \; s_k \le t \le e_k.$$

2. **No overlap:** no two chosen activities overlap, that is,

$$\forall\, k \neq l \in S : \; e_k \le s_l \;\text{ or }\; e_l \le s_k.$$

The *cost* of S is $c(S) = \sum_{k \in S} c_k$.

Goal. Find a feasible schedule S of minimum cost $c(S)$. If no feasible schedule exists, report that the problem has no solution (the algorithm returns ∞).

2 Assumptions

1. **Closed intervals.** Each activity occupies the closed interval $[s_k, e_k]$. A cranky time that coincides with the start or the end of an activity is therefore covered by that activity.

2. **Touching activities do not overlap.** Two activities with $e_k = s_l$ may both be chosen, because they share only an endpoint. Overlap means $e_k > s_l$ and $e_l > s_k$.

3. **Time representation.** Times are numbers measured in a common unit, for example minutes after midnight (so 6 AM = 360 and 10 PM = 1320), and all activities lie in this window. Only comparisons between times are used.

4. **Valid activities.** Every activity has $s_k < e_k$ and a positive fee $c_k > 0$. Each activity can be chosen at most once.

5. **Cranky times.** They are given in any order, and are sorted by the algorithm. A cranky time is covered only by an activity in the chosen schedule.

6. **No cranky times.** If $T = \emptyset$, the empty schedule is feasible with cost 0, so the answer is 0.

7. **No feasible schedule.** If no feasible schedule exists, the answer is ∞.

8. **Cost model.** Comparing and adding two numbers takes constant time. This is used in the complexity analysis.

3 Brute-Force Approach

Enumerate all 2^n subsets of activities, test the no-overlap condition by sorting the subset by end time and comparing neighbours, test the coverage condition by checking each cranky time against each chosen interval, and keep the cheapest feasible subset. For a subset of size $k \le n$, the overlap test costs $O(k \log k)$ and the coverage test $O(mk)$, hence

$$T_{\text{brute}}(n, m) = O\big(2^n \cdot n\,(m + \log n)\big),$$

which is exponential and only usable for tiny inputs. Its role in this project is as a *ground-truth oracle* for validating the efficient algorithm.

```
from itertools import combinations

def no_overlap(chosen):
    chosen=sorted(chosen,key= lambda a:a[1])
    for i in range(1,len(chosen)):
        if chosen[i-1][1]>chosen[i][0]:
            return False
    return True

def all_covered(chosen,cranky):
    for i in cranky:
        covered=False
        for j in range(len(chosen)):
            if i>=chosen[j][0] and i<=chosen[j][1]:
                covered=True
                break
        if not covered:
            return False
    return True

def brute_force(activities,cranky):
    INF=float('inf')
    best=INF
    n=len(activities)

    for k in range(n+1):
        for subset in combinations(activities,k):
            if no_overlap(subset) and all_covered(subset,cranky):
                cost=sum(s[2] for s in subset)
                best=min(best,cost)
    return best
```

4 Key Observations

1. **Feasible schedules are chains.** A feasible schedule has no overlaps, so its activities can be laid out left to right on the timeline. After sorting by end time, they form a chain $i_1 < i_2 < \cdots < i_k$ with $e_{i_p} \le s_{i_{p+1}}$.

2. **Gaps between consecutive activities.** Between two consecutive chosen activities j and i there is a gap (e_j, s_i) during which the child is enrolled in nothing. So *no cranky time may lie strictly inside this gap*:

$$T \cap (e_j, s_i) = \emptyset.$$

   A cranky time exactly at e_j or s_i is fine, since it is covered by j or i.

3. **The two ends of the day.** The same idea applies to the parts before the first and after the last chosen activity.

   - Before the first activity i_1: no cranky time t with $t < s_{i_1}$.
   - After the last activity i_k: no cranky time t with $t > e_{i_k}$.

4. **Global condition becomes local.** Every point of the timeline is either inside a chosen activity (covered), in a gap between two consecutive chosen activities, or before the first / after the last one. So the global condition "every cranky time is covered" becomes a set of local conditions, each involving only the first activity, the last activity, or one pair of consecutive activities. Whether i can be appended to a chain depends only on the last activity j of that chain, not on the earlier history. This is what makes a DP over chains possible.

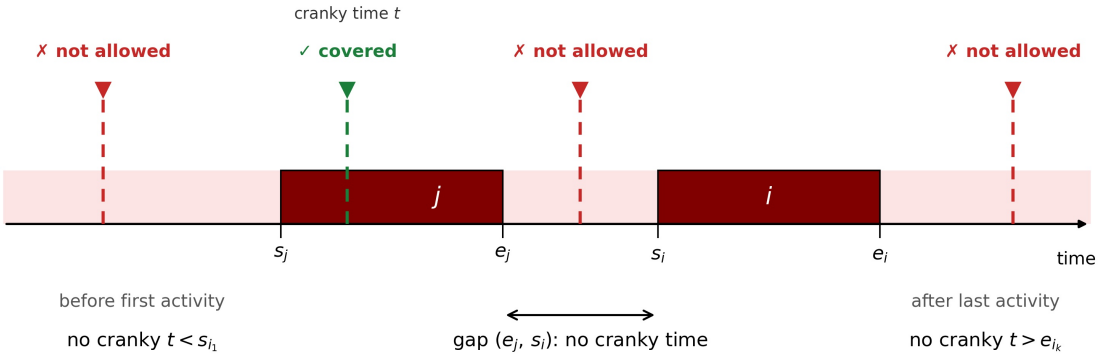


Figure 1: A cranky time inside a chosen activity is covered. A cranky time inside a gap, before the first activity, or after the last activity makes the schedule infeasible.

5 Definitions

Sort the activities by end time, $e_1 \le e_2 \le \cdots \le e_n$; all indices below refer to this order. For each activity j let

$$\mathrm{nxt}(j) = \min\{t \in T : t > e_j\}$$

be the first cranky time strictly after j finishes ($\mathrm{nxt}(j)$ is undefined, `None` in the code, if there is none). Define three predicates:

$$\begin{aligned}
\mathrm{first_ok}(i) &\iff \text{no } t \in T \text{ with } t < s_i && (i \text{ may start the chain}),\\
\mathrm{last_ok}(i) &\iff \text{no } t \in T \text{ with } t > e_i && (i \text{ may end the chain}),\\
\mathrm{compatible}(j, i) &\iff e_j \le s_i \ \text{ and } \ \mathrm{nxt}(j) \ge s_i && (i \text{ may directly follow } j).
\end{aligned}$$

Note that $\mathrm{nxt}(j) \geq s_i$ (or $\mathrm{nxt}(j)$ undefined) says exactly that no cranky time lies in the open gap (e_j, s_i); a cranky time equal to s_i is covered by i itself.

The implementations of the three predicates:

```
def first_ok(i,activities,cranky):
    for t in cranky:
        if t<activities[i][0]:
            return False
    return True

def compatible(j,i,activities,nxt):
    if activities[j][1]>activities[i][0]:
        return False
    if nxt[j] is None or nxt[j]>=activities[i][0]:
        return True
    return False

def last_ok(i,activities,cranky):
    for t in cranky:
        if t>activities[i][1]:
            return False
    return True
```

Definition 2 (The value $f(i)$)**.** For every activity i, $f(i)$ is the minimum total fee over all sets S of pairwise non-overlapping activities such that

- i is the last activity of S (the one ending latest), and
- every cranky time $t \leq e_i$ lies inside some activity of S.

If no such set exists, $f(i) = \infty$.

6 The Recurrence

$$f(i) = c_i + \min\Bigg(\underbrace{\begin{cases} 0 & \text{if first_ok}(i) \\ \infty & \text{otherwise} \end{cases}}_{i \text{ is the first activity}}, \quad \underbrace{\min_{\substack{j<i \\ \mathrm{compat}(j,i)}} f(j)}_{j \text{ directly precedes } i} \Bigg),$$

and the final answer is

$$\mathrm{OPT} = \min\{f(i) : \text{last_ok}(i)\} \quad (\text{and } 0 \text{ if } T = \emptyset).$$

7 Algorithm

The program sorts the activities by end time, precomputes nxt, fills f from left to right using the recurrence, and returns the minimum of $f(i)$ over the activities with last_ok(i).

```
def schedule_activities(activities,cranky):
    if len(cranky)==0:
        return 0

    activities=sorted(activities, key=lambda a: a[1])
    cranky=sorted(cranky)
    n=len(activities)

    nxt=[]
    for j in range(n):
        first_after=None
        for t in cranky:
            if t>activities[j][1]:
                first_after=t
                break
        nxt.append(first_after)

    INF=float('inf')
    f=[INF]*n
    ans=INF

    for i in range(n):
        if first_ok(i,activities,cranky):
            best= 0
        else:
            best=INF

        for j in range(i):
            if compatible(j,i,activities,nxt):
                best=min(best,f[j])
        f[i]=activities[i][2]+best

        if last_ok(i,activities,cranky):
            ans=min(ans,f[i])

    return ans
```

8 Proof of Correctness

Lemma 1 (Chain order)**.** *If S is a set of pairwise non-overlapping activities, ordering it by end time gives $i_1 < \cdots < i_k$ with $e_{i_p} \le s_{i_{p+1}}$ for all p.*

Proof. For non-overlapping a, b either $e_a \le s_b$ or $e_b \le s_a$. In the first case $e_a \le s_b < e_b$, so a precedes b in end-time order. Hence consecutive members in end-time order satisfy $e_{i_p} \le s_{i_{p+1}}$. □

Lemma 2 (Gap criterion)**.** *Let $i_1 < \cdots < i_k$ be a chain. Every $t \in T$ with $t \le e_{i_k}$ is covered iff (1) no $t \in T$ has $t < s_{i_1}$, and (2) for every $p < k$ no $t \in T$ lies in the open interval $(e_{i_p}, s_{i_{p+1}})$.*

Proof. The half-line $(-\infty, e_{i_k}]$ is the disjoint union (up to endpoints) of $(-\infty, s_{i_1})$, the intervals $[s_{i_p}, e_{i_p}]$ and the gaps $(e_{i_p}, s_{i_{p+1}})$. Points of the middle intervals are covered; points of the other regions lie in no chosen interval. So all $t \le e_{i_k}$ are covered iff T avoids those uncovered regions. □

Lemma 3 (Compatibility)**.** compat(j, i) *holds iff* $e_j \le s_i$ *and* $T \cap (e_j, s_i) = \emptyset$.

Proof. nxt(j) is the smallest cranky time exceeding e_j. The open gap (e_j, s_i) contains no cranky time iff that smallest one is absent or is $\ge s_i$. □

Theorem 1. *For each* i*, the value* $f(i)$ *computed by the recurrence equals the minimum defined above, and* OPT *is the minimum cost of a feasible schedule.*

Proof. f *by induction on* i *(in sorted order).* Let S be an optimal set for i (last activity i, covering all $t \le e_i$).

Case 1: $S = \{i\}$. Then every $t \le e_i$ must lie in $[s_i, e_i]$, i.e. no $t < s_i$: first_ok(i) holds and the cost is c_i.

Case 2: $|S| \ge 2$. Let j be the predecessor of i in S (Lemma 1), so $j < i$. By Lemma 2 applied to S, $T \cap (e_j, s_i) = \emptyset$, hence compat$(j, i)$. Moreover $S \setminus \{i\}$ is a valid candidate for j: it is non-overlapping, ends with j, and covers every $t \le e_j$ (the only point of $[s_i, e_i]$ that is $\le e_j$ is $s_i = e_j$, which j also covers). Therefore cost$(S) \ge c_i + f(j)$.

Conversely, take any compatible $j < i$ and an optimal set S_j for j (by the induction hypothesis of cost $f(j)$). Then $S_j \cup \{i\}$ is non-overlapping: S_j is itself non-overlapping, and every activity $a \in S_j$ ends no later than j, so $e_a \le e_j \le s_i$ and a does not overlap i. It also covers every $t \le e_i$: those $t \le e_j$ by S_j, those in (e_j, s_i) do not exist by compatibility, and those in $[s_i, e_i]$ by i. So $f(i) \le c_i + f(j)$. Likewise if first_ok(i) then $\{i\}$ is a valid candidate of cost c_i. Combining both directions gives the recurrence.

Final answer. Let S^* be an optimal feasible schedule ($T \ne \emptyset$) and i its last activity. All its intervals end by e_i, so a cranky time $t > e_i$ could not be covered; hence last_ok(i) holds, and S^* is a candidate for $f(i)$, so cost$(S^*) \ge f(i)$. Conversely, for any i with last_ok(i) the set realising $f(i)$ covers all $t \le e_i$ and there is no $t > e_i$, so it is feasible with cost $f(i)$. Thus OPT $= \min_{\text{last_ok}(i)} f(i)$. □

9 Complexity Analysis

Let n be the number of activities and m the number of cranky times. We assume that comparing and adding two numbers takes constant time.

9.1 Dynamic programming

Step	Cost
Sort activities and cranky times	$O(n \log n + m \log m)$
Compute nxt(j) for all j (linear scan of T)	$O(nm)$
first_ok(i), last_ok(i) for all i	$O(nm)$
compat(j, i) for all pairs, $O(1)$ each thanks to nxt	$O(n^2)$
Total time	$\boldsymbol{O(n^2 + nm + m \log m)}$
Space	$O(n + m)$

Without precomputing nxt, each compatibility test would scan T and the algorithm would cost $O(n^2 m)$; the precomputation removes the factor m from the quadratic term.

9.2 Brute force

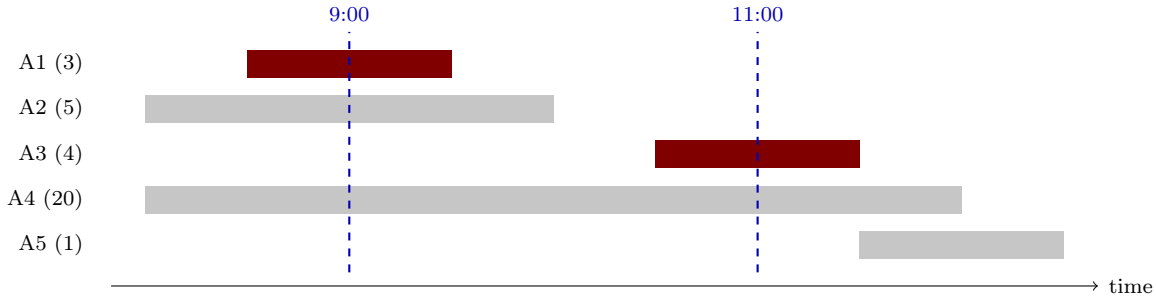
There are 2^n subsets. For a subset of size $k \le n$, the overlap test costs $O(k \log k)$ and the coverage test $O(mk)$, so the total time is $O\big(2^n \cdot n\,(m + \log n)\big)$ and the space is $O(n)$. Compared to this, the DP is polynomial.

10 Worked Example

Times are in minutes after midnight and the cranky times are $T = \{540, 660\}$ (9:00 AM and 11:00 AM). The activities, sorted by end time:

i	(s_i, e_i)	c_i	first_ok	last_ok	nxt	$f(i)$
1	(510,570)	3	yes	no	660	3
2	(480,600)	5	yes	no	660	5
3	(630,690)	4	no	yes	–	7
4	(480,720)	20	yes	yes	–	20
5	(690,750)	1	no	yes	–	8

For example $f(3) = 4 + \min(f(1), f(2)) = 7$ (activities $1, 2$ are both compatible with 3, since nxt $= 660 \ge 630$), and $f(5) = 1 + f(3) = 8$ (activities $1, 2$ are *not* compatible with 5: nxt $= 660 < 690$ leaves 660 uncovered; activity 4 overlaps 5). The answer is $\min(f(3), f(4), f(5)) = \mathbf{7}$, obtained by activities 1 and 3.



Chosen activities in red; dashed lines are the cranky times.

11 Implementation and Testing

11.1 Implementation

The implementation is available at https://github.com/SaiShashankG/Kid_Cranky_Time.

- `optimized.py`: the DP algorithm (function `schedule_activities(activities, cranky)`).
- `brute_force.py`: the exhaustive oracle (function `brute_force(activities, cranky)`).
- `notes.ipynb`: step-by-step development and the randomised test.
- `README.md`: how to run the code. It requires Python 3.8+ and uses only the standard library. Input: `activities` is a list of `(start, end, fee)` tuples and `cranky` a list of times in the same unit. The function returns the minimum total fee, `0` if `cranky` is empty, or `inf` if no valid schedule exists.

11.2 Test cases

Both implementations return the same results on the provided cases:

Test	Brute force	DP
Worked example above	7	7
$\{(8,9,2),(10,11,3),(12,13,4)\}$, $T=\{8.5,10.5,12.5\}$	9	9
Same activities, $T=\emptyset$	0	0

11.3 Randomised testing

In addition, a randomised test generated 2000 random instances ($n \le 8$ activities with integer endpoints in $[0,24]$, fees in $[1,10]$, 1–4 cranky times) and compared the DP against brute force; the two always agreed, including the infeasible cases where both return ∞.

```
import random

def random_test(trials=2000):
    for _ in range(trials):
        n = random.randint(1, 8)
        activities = []
        for _ in range(n):
            s = random.randint(0, 20)
            e = random.randint(s + 1, 24)
            fee = random.randint(1, 10)
            activities.append((s, e, fee))
        cranky = [random.randint(0, 24) for _ in range(random.randint(1, 4))]
        if schedule_activities(activities,cranky)!=brute_force(activities,cranky):
            return False
    return True
```

12 Conclusion

The problem is a weighted-interval-style covering problem. Sorting by end time and capturing "no cranky time falls in a gap" with the precomputed nxt array reduces it to a clean chain DP running in $O(n^2 + nm + m \log m)$ time. The total time is $O(n^2 + nm + m \log m)$. Since $\log m \le n$ whenever $m \le 2^n$, which always holds for realistic inputs, $m \log m = O(nm)$ and the bound simplifies to $O(n^2 + nm)$, proved correct above and validated against an exponential brute-force oracle.